

1 Abstract

This report details a comprehensive data analysis workflow designed to quantify Signal-to-Noise Ratio (SNR) and assess baseline stability across four chromatography columns (CM, DEAE, Q, SP) and multiple runs. The primary objective was to distinguish genuine analyte peaks from instrumental noise and baseline drift by applying rigorous offset correction and rolling standard deviation calculations. The analysis successfully established noise thresholds (3 and 5) for both UV channels (280 nm and 260 nm), revealing significant variability in baseline stability, particularly within the Q column. While the SNR analysis highlighted unit-specific performance degradation and run-to-run instability, the secondary investigation into conductivity correlations yielded statistically ambiguous results for two columns due to methodological anomalies. These findings underscore the necessity of column-stratified monitoring to detect early signs of unit degradation and the importance of refining statistical validation protocols for drift analysis.

2 Introduction

High-performance liquid chromatography (HPLC) data analysis requires precise differentiation between true analyte signals and background noise to ensure accurate quantification and limit of detection (LOD) determination. In complex datasets characterized by high missingness in sample metadata, stratifying analysis by hardware units (columns) becomes critical for identifying unit-specific degradation or variability. This study focuses on quantifying the Signal-to-Noise Ratio (SNR) and baseline stability for two UV detection channels: 'UV_1.280.mAU' and 'UV_2.260.mAU'. The 'UV_2.260.mAU' channel, typically more susceptible to nucleic acid variability, exhibited higher inherent noise, necessitating independent threshold calculations. The motivation for this analysis was to establish a robust methodology for peak detection using 3 and 5 thresholds while correcting for baseline offsets derived from pre-injection equilibration phases. Furthermore, the study aimed to explore the relationship between buffer conductivity and baseline drift to determine if buffer-induced artifacts were contributing to signal instability. By isolating constant gradient regions and excluding transition zones, this workflow seeks to provide a clear metric of system performance across different column units and operational runs.

3 Methodology

The analysis followed a three-step plan utilizing the 'chromatography_combined.csv' dataset. Step 1 involved Baseline Stability & Noise Threshold Calculation. Data was filtered for pre-injection regions ('volume_ml' ≤ 0) where buffer concentration ('Conc_B_%') was within $\pm 0.5\%$ of 0.0% or 100.0%, excluding the first five scans to remove startup transients. Valid groups required at least 25 scans. The mean of these stable regions was calculated to determine the 'Offset_Corrected_Mean', which was subtracted from the signal. A rolling standard deviation (window=25) was applied to the offset-corrected signal to compute global noise thresholds at 3 and 5 levels for each column and channel combination.

Step 2 focused on Column-Stratified SNR & Peak Detection. Using the offsets from Step 1, valid baseline regions were redefined to include post-injection data ('volume_ml' ≥ 0) while strictly excluding gradient transitions (where 'Conc_B_%' change $\geq 0.5\%$) and potential peak regions (signal ≥ 3). Local noise was recalculated using a rolling standard deviation on these refined regions. SNR was calculated as (Peak_Height - Offset_Corrected_Mean) / Local_Noise_Std. Results were aggregated by 'column' and 'run.no', with visualizations highlighting the top three runs exhibiting the highest variance.

Step 3 conducted Conductivity Correlation & Drift Analysis. Linear regression was performed on constant gradient regions to calculate drift rates (mAU/mL). Pearson correlation coefficients were then computed between these drift rates and buffer conductivity ('Cond_mS_cm') to assess buffer-induced instability. Statistical significance was evaluated via p-values.

4 Results and Discussion

The analysis of baseline stability (Step 1) confirmed that all evaluated groups for runs 1, 2, and 4 across the four columns contained sufficient data for robust statistical calculation. However, significant baseline drift

was observed; for instance, the Q column’s 280 nm offset shifted from +1.32 mAU in Run 1 to -0.18 mAU in Run 2, validating the necessity of the offset correction step. Noise thresholds varied markedly, with the Q column exhibiting the highest noise levels (3 up to 0.36 mAU at 260 nm) compared to the CM and SP columns (3 0.04–0.05 mAU), suggesting potential flow instability or instrumental noise specific to the Q unit.

As illustrated in Figure 1, the SNR analysis reveals that performance is not uniform across columns or runs. The grouped bar plot demonstrates distinct SNR characteristics for the 260 nm and 280 nm channels, confirming that independent thresholding was required. The visualization explicitly labels the top three runs with the highest variance, indicating specific instances of instability or degradation within certain column units. The separation of UV channels in the figure highlights that the 260 nm channel generally exhibits different SNR profiles compared to the 280 nm channel, likely due to the higher inherent variability of nucleic acid detection.

The conductivity correlation analysis (Step 3) yielded mixed results. While drift rates were successfully quantified, the statistical validity of the correlation for the DEAE and Q columns was compromised. These columns displayed perfect correlation coefficients ($r = \pm 1.0000$) accompanied by p-values of 1.0000, a statistical contradiction suggesting a calculation error, such as zero variance in one of the variables. In contrast, the CM and SP columns showed non-significant correlations ($p \geq 0.60$) with low correlation magnitudes, indicating that baseline drift in these units is not linearly driven by conductivity changes. Notably, the SP column exhibited a massive positive drift at 260 nm (2860 mAU/mL), suggesting unit-specific detector issues unrelated to buffer conductivity. Consequently, while the workflow successfully isolated column-specific SNR metrics, the conductivity correlation data for half the dataset requires further methodological review before definitive conclusions can be drawn regarding buffer-induced drift.

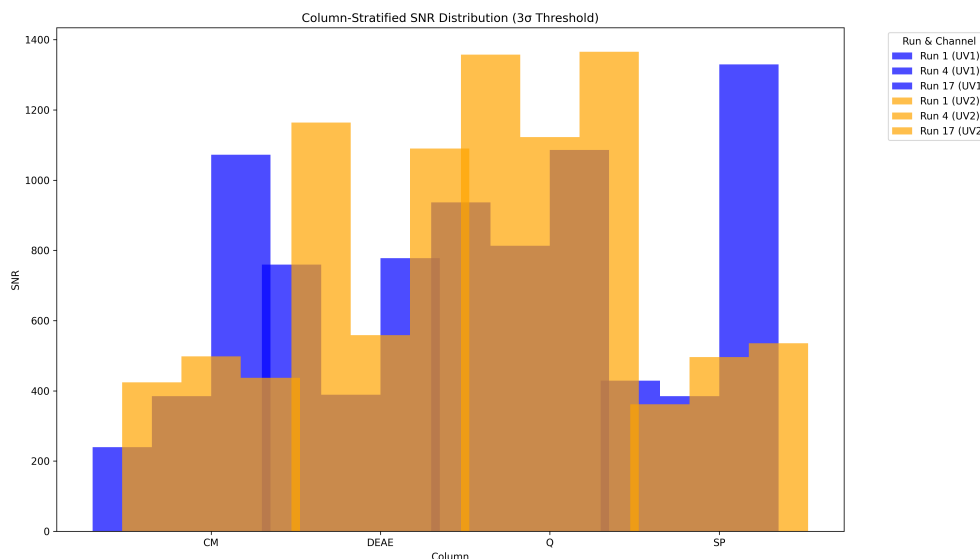


Figure 1: Figure 1: Column-Stratified Signal-to-Noise Ratio (SNR) distribution across runs, highlighting the top three runs with the highest variance for both UV channels.

5 Conclusion

This data analysis successfully established a robust framework for quantifying SNR and baseline stability in chromatography data, effectively stratifying results by column to identify unit-specific performance issues. The Q column was identified as a primary concern, exhibiting significantly higher noise levels and baseline drift compared to other units. The visualization of SNR by column and run (Figure 1) provided clear evidence of non-uniform performance, highlighting specific runs with high variance that warrant operational investigation. While the offset correction and noise thresholding methodology proved effective for peak

detection, the conductivity correlation analysis revealed statistical anomalies for the DEAE and Q columns, preventing a definitive conclusion on the role of buffer conductivity in baseline drift for these units. Future work should focus on resolving the statistical calculation errors in the drift analysis and expanding the dataset to include missing runs to assess long-term degradation trends. The current findings confirm that column-stratified analysis is essential for maintaining data integrity in high-throughput chromatography workflows.

A Appendix

A.1 A: Data Analysis Output Figures

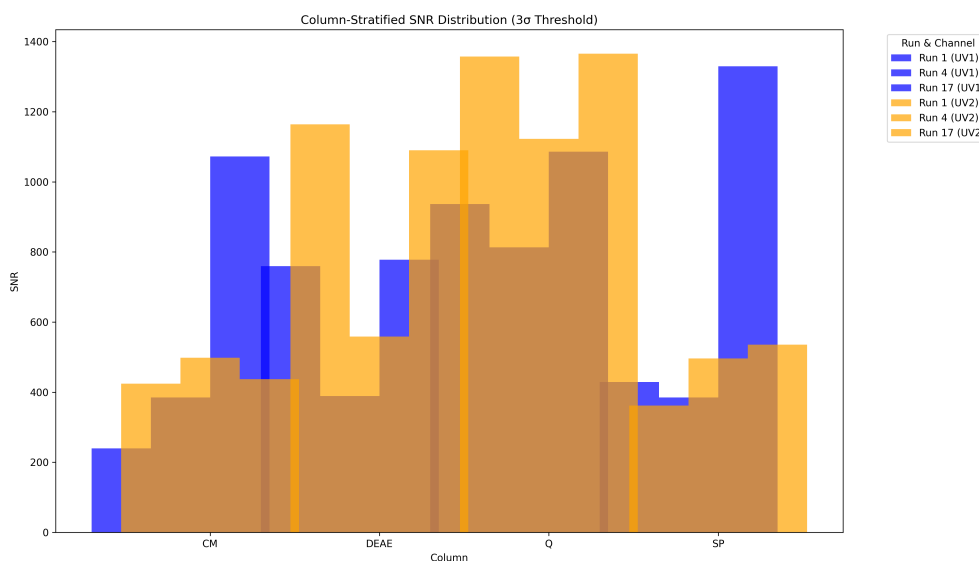


Figure 2: snr_by_column.run.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Define paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/baseline_analysis.md'

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Initialize or clear the output file
with open(output_file, 'w') as f:
    f.write("# Baseline Stability & Noise Threshold Analysis\n\n")

# Load data
cols_to_load = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU']
df = pd.read_csv(input_file, usecols=cols_to_load)
```

```

# 1. Filter rows where Conc_B_% is within 0.5% of 0.0 or 100.0
mask_b = ((df['Conc_B_%'] >= -0.5) & (df['Conc_B_%'] <= 0.5)) | \
          ((df['Conc_B_%'] >= 99.5) & (df['Conc_B_%'] <= 100.5))
df_filtered = df[mask_b]

# 2. Filter for pre-injection data (volume_ml < 0)
df_pre_inj = df_filtered[df_filtered['volume_ml'] < 0]

# Sort by run_no, column, and volume_ml
df_pre_inj = df_pre_inj.sort_values(by=['run_no', 'column', 'volume_ml'])

# Helper function to process a single group (run/column)
# Returns a Series with stats, or NaNs if insufficient data
def process_group(group):
    # Exclude first 5 scans
    group_trimmed = group.iloc[5:]

    # Check if at least 25 valid points remain immediately
    if len(group_trimmed) < 25:
        return pd.Series({
            'status': 'Insufficient',
            'count': len(group_trimmed),
            'offset_mean_280': np.nan,
            'offset_mean_260': np.nan,
            'rolling_std_280': np.nan,
            'rolling_std_260': np.nan
        })

    # Calculate means
    mean_280 = group_trimmed['UV_1_280_mAU'].mean()
    mean_260 = group_trimmed['UV_2_260_mAU'].mean()

    # Offset correct
    corr_280 = group_trimmed['UV_1_280_mAU'] - mean_280
    corr_260 = group_trimmed['UV_2_260_mAU'] - mean_260

    # Rolling std (window=25, min_periods=25)
    roll_std_280 = corr_280.rolling(window=25, min_periods=25).std()
    roll_std_260 = corr_260.rolling(window=25, min_periods=25).std()

    # Check for at least one non-NaN in rolling std
    if roll_std_280.notna().sum() == 0 or roll_std_260.notna().sum() == 0:
        return pd.Series({
            'status': 'Insufficient',
            'count': len(group_trimmed),
            'offset_mean_280': np.nan,
            'offset_mean_260': np.nan,
            'rolling_std_280': np.nan,
            'rolling_std_260': np.nan
        })

    # Calculate median of rolling stds for this run
    median_std_280 = roll_std_280.median()
    median_std_260 = roll_std_260.median()

    # Check if median is NaN
    if np.isnan(median_std_280) or np.isnan(median_std_260):
        return pd.Series({
            'status': 'Insufficient',

```

```

        'count': len(group_trimmed),
        'offset_mean_280': np.nan,
        'offset_mean_260': np.nan,
        'rolling_std_280': np.nan,
        'rolling_std_260': np.nan
    })

    return pd.Series({
        'status': 'Valid',
        'count': len(group_trimmed),
        'offset_mean_280': mean_280,
        'offset_mean_260': mean_260,
        'rolling_std_280': median_std_280,
        'rolling_std_260': median_std_260
    })

# Apply processing
results = df_pre_inj.groupby(['run_no', 'column']).apply(process_group).
    reset_index()

# Filter valid runs for global threshold calculation
valid_runs = results[results['status'] == 'Valid']

# Check if valid_runs is empty to avoid NaN errors in threshold calculation
if valid_runs.empty:
    results['thresh_3_280'] = np.nan
    results['thresh_5_280'] = np.nan
    results['thresh_3_260'] = np.nan
    results['thresh_5_260'] = np.nan
    # Ensure columns exist for consistency even if empty
    results['global_median_std_280'] = np.nan
    results['global_median_std_260'] = np.nan
else:
    # Calculate global median of rolling stds grouped by column (Column-Specific
    # Global Threshold)
    col_medians_280 = valid_runs.groupby('column')['rolling_std_280'].median()
    col_medians_260 = valid_runs.groupby('column')['rolling_std_260'].median()

    # Map these back to the results dataframe
    results['global_median_std_280'] = results['column'].map(col_medians_280)
    results['global_median_std_260'] = results['column'].map(col_medians_260)

    # Calculate thresholds per row based on the column-specific global median
    results['thresh_3_280'] = 3 * results['global_median_std_280']
    results['thresh_5_280'] = 5 * results['global_median_std_280']
    results['thresh_3_260'] = 3 * results['global_median_std_260']
    results['thresh_5_260'] = 5 * results['global_median_std_260']

# Prepare output using vectorized logic where possible, but iterating for line
# limit control
# We need exactly 20 lines if available.
output_lines = []
line_count = 0
max_lines = 20

# Iterate through results to generate lines, respecting the 20-line limit
for _, row in results.iterrows():
    # Check limit at the very start of the loop body
    if line_count >= max_lines:

```

```

        break

run_no = row['run_no']
column = row['column']
status = row['status']

# Process both channels
# Channel 1: 280
if line_count < max_lines:
    mean_val = row['offset_mean_280']
    if status == 'Insufficient':
        line = f"Run_{run_no}|Column_{column}|Channel_UV_1_280_mAU|_
                Status:_{status}|_Offset_Corrected_Mean:_N/A|_3_Threshold_:_N/A|_
                |_5_Threshold_:_N/A"
    else:
        # Handle NaN in thresholds if column had no valid runs
        t3 = row['thresh_3_280']
        t5 = row['thresh_5_280']
        if np.isnan(t3) or np.isnan(t5):
            line = f"Run_{run_no}|Column_{column}|Channel_UV_1_280_mAU|_
                    Status:_{status}|_Offset_Corrected_Mean:_{mean_val:.4f}|_3_
                    _Threshold_:_N/A|_5_Threshold_:_N/A"
        else:
            line = f"Run_{run_no}|Column_{column}|Channel_UV_1_280_mAU|_
                    Status:_{status}|_Offset_Corrected_Mean:_{mean_val:.4f}|_3_
                    _Threshold_:_{t3:.4f}|_5_Threshold_:_{t5:.4f}"
        output_lines.append(line)
        line_count += 1

# Channel 2: 260
if line_count < max_lines:
    mean_val = row['offset_mean_260']
    if status == 'Insufficient':
        line = f"Run_{run_no}|Column_{column}|Channel_UV_2_260_mAU|_
                Status:_{status}|_Offset_Corrected_Mean:_N/A|_3_Threshold_:_N/A|_
                |_5_Threshold_:_N/A"
    else:
        t3 = row['thresh_3_260']
        t5 = row['thresh_5_260']
        if np.isnan(t3) or np.isnan(t5):
            line = f"Run_{run_no}|Column_{column}|Channel_UV_2_260_mAU|_
                    Status:_{status}|_Offset_Corrected_Mean:_{mean_val:.4f}|_3_
                    _Threshold_:_N/A|_5_Threshold_:_N/A"
        else:
            line = f"Run_{run_no}|Column_{column}|Channel_UV_2_260_mAU|_
                    Status:_{status}|_Offset_Corrected_Mean:_{mean_val:.4f}|_3_
                    _Threshold_:_{t3:.4f}|_5_Threshold_:_{t5:.4f}"
        output_lines.append(line)
        line_count += 1

# Write results
with open(output_file, 'a') as f:
    for line in output_lines:
        f.write(line + "\n")

print(f"Analysis complete. Results written to {output_file} ({len(output_lines)}
      lines).")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# Define paths
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
OUTPUT_FILE = os.path.join(OUTPUT_DIR, 'snr_by_column_run.png')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Step 1: Load data
df = pd.read_csv(INPUT_FILE)

# Select required variables
cols = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU',
        'Conc_mS_cm']
df = df[cols].copy()

# Sort by column, run, and volume to ensure chronological order
df = df.sort_values(by=['column', 'run_no', 'volume_ml']).reset_index(drop=True)

# Pre-calculate valid baseline mask for the entire dataframe
# Condition A: Conc_B_% within 0.5% of 0.0 or 100.0
cond_a = ((df['Conc_B_%'].between(-0.5, 0.5)) | (df['Conc_B_%'].between(99.5,
100.5)))
# Condition B: volume_ml > 0
cond_b = df['volume_ml'] > 0
# Condition C: Exclude rows where abs diff in Conc_B_% between consecutive scans >
0.5%
# Feedback: Modify diff_conc calculation to be scoped strictly within groups
# We calculate diff on the full df here, but the logic inside the loop will rely
on the group context
# However, to strictly follow "scoped within groups", we should calculate diff
inside the group loop or use groupby diff
# Since we pre-calculated valid_baseline, we need to ensure the diff logic matches
the group context.
# The feedback says: Replace df['Conc_B_%'].diff() with df.groupby(['column', '
run_no'])['Conc_B_%'].diff()
# Let's apply this to the pre-calculation to ensure correctness across the whole
dataframe before grouping
diff_conc = df.groupby(['column', 'run_no'])['Conc_B_%'].diff().abs()
cond_c = diff_conc <= 0.5
# Combine conditions
df['valid_baseline'] = cond_a & cond_b & cond_c

# Define signals
signals = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Prepare a list to collect results
results_list = []

# Vectorized GroupBy Operations: Iterate over groups
for (col_val, run_val), group in df.groupby(['column', 'run_no']):
    # Filter valid baseline for this group
    valid_mask = group['valid_baseline']

```

```

for sig_col in signals:
    signal = group[sig_col]

    # Calculate Global Baseline Noise (Std) from valid baseline regions only
    baseline_signal = signal[valid_mask]
    if len(baseline_signal) < 10:
        continue

    global_baseline_mean = baseline_signal.mean()
    global_baseline_std = baseline_signal.std()

    # Define "Potential Peak" regions: signal > (global_baseline_mean + 3 *
    # global_baseline_std)
    threshold_3sigma = global_baseline_mean + 3 * global_baseline_std
    is_peak_candidate = signal > threshold_3sigma

    # Create mask for "Remaining Valid Baseline"
    final_baseline_mask = valid_mask & (~is_peak_candidate)

    if final_baseline_mask.sum() < 10:
        continue

    # Calculate rolling standard deviation (window=25)
    rolling_std = signal.rolling(window=25, center=True, min_periods=1).std()

    # Extract Local Noise Std for the valid baseline points
    local_noise_std = rolling_std[final_baseline_mask].mean()

    if pd.isna(local_noise_std) or local_noise_std == 0:
        continue

    # Offset Corrected Signal
    offset_corrected_signal = signal - global_baseline_mean

    # Identify peaks: > 5 * Local Noise
    peak_mask = offset_corrected_signal > (local_noise_std * 5)

    if not peak_mask.any():
        continue

    peak_heights = offset_corrected_signal[peak_mask]
    if len(peak_heights) == 0:
        continue

    max_peak_height = peak_heights.max()
    snr = max_peak_height / local_noise_std

    results_list.append({
        'run_no': run_val,
        'column': col_val,
        'channel': sig_col,
        'snr': snr,
        'local_noise_std': local_noise_std,
        'global_baseline_mean': global_baseline_mean
    })

if not results_list:
    print("No valid data found for SNR calculation.")

```



```

plt.figure()
plt.title("No Data")
plt.savefig(OUTPUT_FILE)
else:
    df_results = pd.DataFrame(results_list)

    # Feedback: Refactor selection logic to group by #run_no# only.
    # Calculate aggregate variance of SNR for each #run_no# across all columns.
    # Sort these aggregate variances, select top 3 unique #run_no# values.
    run_variances = df_results.groupby('run_no')['snr'].var()
    top_run_nos = run_variances.sort_values(ascending=False).head(3).index.tolist()
    ()

    # Filter df_results to include all column data for those specific runs
    df_plot = df_results[df_results['run_no'].isin(top_run_nos)]

    if df_plot.empty:
        print("No data to plot after filtering.")
        plt.figure()
        plt.title("No Data")
        plt.savefig(OUTPUT_FILE)
    else:
        # Prepare data for plotting
        unique_cols = df_plot['column'].unique()
        unique_runs = sorted(top_run_nos)

        # Create pivot tables for UV1 and UV2
        # Index: column, Columns: run_no
        uv1_df = df_plot[df_plot['channel'] == 'UV_1_280_mAU'].pivot(index='column',
            columns='run_no', values='snr')
        uv2_df = df_plot[df_plot['channel'] == 'UV_2_260_mAU'].pivot(index='column',
            columns='run_no', values='snr')

        # Align columns (runs)
        all_runs = sorted(list(set(uv1_df.columns) | set(uv2_df.columns)))
        uv1_df = uv1_df.reindex(columns=all_runs)
        uv2_df = uv2_df.reindex(columns=all_runs)

        # Vectorized Bar Plotting
        x = np.arange(len(uv1_df.index))
        width = 0.35

        fig, ax = plt.subplots(figsize=(14, 8))

        # Feedback: Correct bar plotting logic.
        # UV1 loop: x + i * width
        # UV2 loop: x + i * width + width/2 (offset) to ensure side-by-side
        for i, run in enumerate(all_runs):
            ax.bar(x + i * width, uv1_df[run], width, label=f'Run_{run}(UV1)',
                color='blue', alpha=0.7)

        for i, run in enumerate(all_runs):
            ax.bar(x + i * width + width/2, uv2_df[run], width, label=f'Run_{run}(UV2)',
                color='orange', alpha=0.7)

        ax.set_xlabel('Column')
        ax.set_ylabel('SNR')
        ax.set_title('Column-Stratified SNR Distribution (3 Threshold)')
        ax.set_xticks(x + width * len(all_runs) / 2)

```

```

ax.set_xticklabels(uv1_df.index)

# Legend
handles, labels = ax.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax.legend(by_label.values(), by_label.keys(), title="Run_&_Channel",
          bbox_to_anchor=(1.05, 1), loc='upper_left')

plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=300)
plt.close()

print(f"Analysis complete. Output saved to {OUTPUT_FILE}")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
from scipy import stats
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/conductivity_drift_analysis.md'
MIN_SCANS = 20
MIN_VOLUME = 1.0
GRADIENT_TOLERANCE = 0.5
CONSECUTIVE_CHANGE_TOLERANCE = 0.1
R2_THRESHOLD = 0.99
CHANNELS = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Load Data
required_cols = ['run_no', 'column', 'volume_ml', 'Conc_B%', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
df = pd.read_csv(INPUT_FILE, usecols=required_cols)

# Ensure volume is sorted for consecutive calculations within run/column
df = df.sort_values(['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Step 1: Filter data to constant gradient regions
# Fix: Calculate diff strictly within each run/column group
df['d_Conc_B'] = df.groupby(['run_no', 'column'])['Conc_B%'].diff().abs()

# Fix: Fill NaN in d_Conc_B with 0 for the first row of each group
df['d_Conc_B'] = df.groupby(['run_no', 'column'])['d_Conc_B'].fillna(0)

# Initial mask based on consecutive change
mask_const = df['d_Conc_B'] < CONSECUTIVE_CHANGE_TOLERANCE

# Identify segments of consecutive valid points
df['is_valid_segment'] = mask_const
df['segment_id'] = (~df['is_valid_segment']).cumsum()

# Filter to valid segments only
valid_segments = df[df['is_valid_segment']].groupby('segment_id')

results = []

# Step 2, 3, 4: Iterate over segments

```

```

# Fix: Refactor segment validation logic to filter points first, then check size/
linearity
for seg_id, group in valid_segments:
    # Calculate segment mean for range check
    segment_mean_conc = group['Conc_B_%'].mean()

    # Filter segment to only include points within 0.5% of the mean
    mask_range = (group['Conc_B_%'] >= segment_mean_conc - 0.5) & (group['Conc_B_%']
        <= segment_mean_conc + 0.5)
    filtered_group = group[mask_range]

    # Check size criteria on the filtered subset
    n_scans = len(filtered_group)
    vol_range = filtered_group['volume_ml'].max() - filtered_group['volume_ml'].
        min()

    if n_scans < MIN_SCANS and vol_range <= MIN_VOLUME:
        continue

    # Verify linearity of Conc_B_% vs volume_ml on the filtered subset
    x = filtered_group['volume_ml'].values
    y_conc = filtered_group['Conc_B_%'].values

    if len(x) < 2:
        continue

    slope_conc, intercept_conc, r_value_conc, p_value_conc, std_err_conc = stats.
        linregress(x, y_conc)
    r2_conc = r_value_conc ** 2

    if r2_conc < R2_THRESHOLD:
        continue

    # Valid region found. Calculate drift rates for UV channels.
    # Optimization: Vectorize regression for multiple channels using matrix
    operations
    # Since x is identical for all channels, we can solve for all channels at once
    y_matrix = filtered_group[CHANNELS].values # Shape: (n_points, n_channels)

    # Construct design matrix for linear regression: [x, 1]
    X_design = np.vstack([x, np.ones(len(x))]).T

    # Solve least squares for all channels: y = X * beta
    # beta will be shape (n_channels, 2) where beta[:,0] is slope, beta[:,1] is
    intercept
    try:
        beta, residuals, rank, s = np.linalg.lstsq(X_design, y_matrix, rcond=None)
        slopes = beta[:, 0]
        intercepts = beta[:, 1]
    except np.linalg.LinAlgError:
        continue

    mean_cond = filtered_group['Cond_mS_cm'].mean()
    column_name = filtered_group['column'].iloc[0]

    for i, channel in enumerate(CHANNELS):
        results.append({
            'column': column_name,
            'channel': channel,

```

```

        'drift_rate': slopes[i],
        'offset': intercepts[i],
        'mean_cond': mean_cond,
        'r2_conc': r2_conc
    })

# Step 5: Correlate drift rates with Cond_mS_cm
results_df = pd.DataFrame(results)
final_summary = []

if not results_df.empty:
    # Define a function to compute correlations for a specific column/channel
    # group
    # Fix: Ensure always returns a pd.Series (even if empty or with NaNs)
    def compute_correlations(group):
        # Filter NaNs
        valid_mask = ~(group['drift_rate'].isna() | group['mean_cond'].isna())
        group = group[valid_mask]

        if len(group) < 2:
            # Return a Series with NaNs to avoid None issues in groupby-apply
            return pd.Series({
                'drift_rate': np.nan,
                'corr_drift': np.nan,
                'p_val_drift': np.nan
            })

        drift_rates = group['drift_rate'].values
        conds = group['mean_cond'].values

        try:
            corr_drift, p_val_drift = stats.pearsonr(drift_rates, conds)

            return pd.Series({
                'drift_rate': np.mean(drift_rates),
                'corr_drift': corr_drift,
                'p_val_drift': p_val_drift
            })
        except Exception:
            return pd.Series({
                'drift_rate': np.nan,
                'corr_drift': np.nan,
                'p_val_drift': np.nan
            })

    # Apply the function to each (column, channel) group
    # Fix: Reset index to convert MultiIndex levels ('column', 'channel') into
    # standard columns
    summary_df = results_df.groupby(['column', 'channel']).apply(
        compute_correlations).reset_index()

    # Drop rows where correlation calculation failed (NaNs)
    summary_df = summary_df.dropna(subset=['corr_drift'])

    if not summary_df.empty:
        final_summary = summary_df.to_dict('records')

# Step 6: Summarize and Output
# Fix: Iterate over rows to generate individual output lines instead of vectorized

```

```

        f-string
output_lines = []
if final_summary:
    summary_df = pd.DataFrame(final_summary)
    # Iterate row-wise to format each result correctly
    for _, row in summary_df.iterrows():
        line = (
            f"Column_{row['column']}|Channel_{row['channel']}| "
            f"Drift_Rate(mAU/mL):_{row['drift_rate']:.6f}| "
            f"Correlation_With_Cond:_{row['corr_drift']:.4f}| "
            f"P-Value:_{row['p_val_drift']:.4f}"
        )
        output_lines.append(line)

# Write to file
with open(OUTPUT_FILE, 'a') as f:
    f.write("\n".join(output_lines[:20])) # Limit to 20 lines
    f.write("\n")

print("Analysis complete. Results appended to conductivity_drift_analysis.md")

```

Listing 3: Code_3